



(19) 대한민국특허청(KR)
(12) 등록특허공보(B1)

(45) 공고일자 2012년02월24일
(11) 등록번호 10-1120398
(24) 등록일자 2012년01월30일

(51) Int. Cl.
G06F 9/30 (2006.01) G06F 9/38 (2006.01)
G06F 9/06 (2006.01)
(21) 출원번호 10-2008-7021561
(22) 출원일자(국제출원일자) 2007년02월05일
심사청구일자 2009년12월08일
(85) 번역문제출일자 2008년09월03일
(65) 공개번호 10-2008-0109743
(43) 공개일자 2008년12월17일
(86) 국제출원번호 PCT/US2007/003313
(87) 국제공개번호 WO 2007/092528
국제공개일자 2007년08월16일
(30) 우선권주장
60/764,955 2006년02월03일 미국(US)
(56) 선행기술조사문헌
US05197140 A1
US19995941981 A1
US6986027 B1

(73) 특허권자
피시, 러셀, 에이치. III
미국, 텍사스 75250, 달라스, 프레스턴 오크스 로드 5400
(72) 발명자
피시, 러셀, 에이치. III
미국, 텍사스 75250, 달라스, 프레스턴 오크스 로드 5400
(74) 대리인
박윤원, 강명구

전체 청구항 수 : 총 16 항

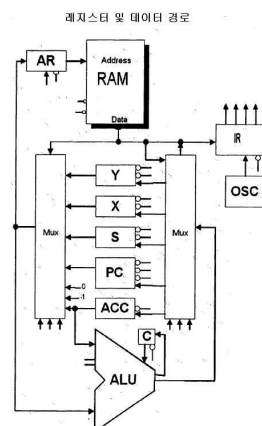
심사관 : 홍경아

(54) 스레드 최적화된 멀티프로세서 구조

(57) 요약

하나의 양태에서, 본 발명은 하나의 시스템을 포함하는데, 상기 시스템은 (a) 단일 칩 상의 다수의 병렬 프로세서와, (b) 상기 칩 상에 위치하고, 상기 프로세서 각각에 의해 액세스 가능한 컴퓨터 메모리를 포함하며, 이때 상기 프로세서 각각은 최소 허용되는(*de minimis*) 인스트럭션 세트를 프로세싱하며, 상기 프로세서 각각은 상기 프로세서의 3가지 이상의 특정 레지스터 각각에 대한 전용 로컬 캐시를 포함한다. 또 다른 양태에서, 본 발명은 하나의 시스템을 포함하는데, 상기 시스템은 (a) 하나의 단일 칩 상의 다수의 병렬 프로세서와, (b) 상기 칩 상에 위치하고, 상기 프로세서 각각에 의해 액세스 가능한 컴퓨터 메모리를 포함하며, 이때, 상기 프로세서 각각은 스레드(thread)-레벨의 병렬 프로세싱에 대하여 최적화된 인스트럭션 세트를 프로세싱한다.

대표도 - 도1



특허청구의 범위

청구항 1

- 단일 칩 상의 복수의 병렬 프로세서, 그리고

- 상기 칩 상에 위치하고, 상기 프로세서 각각에 의해 액세스 가능한 컴퓨터 메모리

를 포함하며, 상기 프로세서 각각은 7개보다 작거나 같은 수의 인스트럭션을 포함하는 기본 인스트럭션 세트를 처리하도록 구성되는 시스템에 있어서,

각각의 프로세서는 복수의 캐시 및 대응하는 어드레싱 레지스터를 포함하여, 각각의 캐시가 특정 어드레싱 레지스터에 연계되고, 각각의 캐시는 대응 어드레싱 레지스터의 최상위 비트가 변화할 때 캐시 로드 작동을 수행하도록 배열되는, 시스템.

청구항 2

제 1 항에 있어서,

로컬 캐시 각각의 크기는 상기 칩 상의 랜덤 액세스 메모리의 하나의 행(row)에 상응하는, 시스템.

청구항 3

제 1 항에 있어서,

대응 캐시를 포함하는 특정 어드레싱 레지스터는 인스트럭션 레지스터, 출발지 레지스터(source register) 및 도착지 레지스터(destination register)를 포함하는, 시스템.

청구항 4

제 1 항에 있어서,

상기 기본 인스트럭션 세트가 7개의 인스트럭션으로 구성되는, 시스템.

청구항 5

제 1 항에 있어서,

상기 프로세서 각각은 단일 스레드(thread)를 처리하도록 작동하는, 시스템.

청구항 6

제 1 항에 있어서,

어큐뮬레이터(accumulator)는 증연산 인스트럭션(increment instruction)을 제외한 모든 인스트럭션에 대한 피연산자(operand)인, 시스템.

청구항 7

제 1 항에 있어서,

상기 기본 인스트럭션 세트 내 각각의 인스트럭션에 대한 도착지는, 항상 각각의 인스트럭션에 대한 피연산자를 저장한 레지스터인, 시스템.

청구항 8

제 1 항에 있어서,

상기 시스템은, 레지스터 간접 자동-증연산 어드레싱 모드로 자동-증연산되도록 구성되는 3개의 추가 레지스터와, 레지스터 간접 자동-감연산 어드레싱 모드로 자동-감연산되도록 구성되는 3개의 추가 레지스터를 포함하는, 시스템.

청구항 9

제 1 항에 있어서,

상기 기본 인스트럭션 세트 내 각각의 인스트럭션은 완료되기 위해 단지 1회의 클럭 사이클만을 필요로 하는, 시스템.

청구항 10

제 1 항에 있어서,

상기 기본 인스트럭션 세트는 어떠한 BRANCH 인스트럭션도, 그리고, 어떠한 JUMP 인스트럭션도 포함하지 않는, 시스템.

청구항 11

제 1 항에 있어서,

상기 기본 인스트럭션 세트 내 각각의 인스트럭션은 8비트 이하의 길이를 갖는, 시스템.

청구항 12

제 1 항에 있어서,

하나의 단일 마스터 프로세서가 상기 병렬 프로세서 각각을 관리하는 것을 담당하는, 시스템.

청구항 13

제 1 항에 있어서,

상기 시스템은,

단일 스레드를 처리하기 위해 복수의 프로세서 중 하나를 할당하는 단계와,

상기 프로세서들에 의해 각각의 할당된 스레드를 처리하는 단계와,

상기 프로세서에 의해 처리되는 각각의 스레드로부터의 결과를 처리하는 단계와,

스레드가 처리된 후 상기 복수의 프로세서 중 하나를 탈-할당하는 단계

를 수행하도록 구성되는, 시스템.

청구항 14

제 4 항에 있어서,

상기 7개의 인스트럭션은,

피연산자의 값을 어큐뮬레이터 내로 로딩하기 위한 인스트럭션과,

어드레스를 어큐뮬레이터 내로 로딩하기 위한 인스트럭션과,

어큐뮬레이터의 값을 어드레스에 저장하기 위한 인스트럭션과,

ADD 인스트럭션과,

AND 인스트럭션과,

XOR 인스트럭션과,

증연산 인스트럭션

인, 시스템.

청구항 15

단일 칩 상의 컴퓨터 메모리 및 복수의 병렬 프로세서를 이용하는 병렬 처리 방법에 있어서,

상기 컴퓨터 메모리는 각각의 프로세서에 의해 액세스가능하고, 각각의 프로세서는 7개보다 작거나 같은 인스트

력선을 포함하는 기본 인스트럭션 세트를 처리하도록 구성되는, 상기 병렬 처리 방법에 있어서,
 각각의 프로세서에서 복수의 어드레싱 레지스터 각각에 대해 캐시를 할당하여, 각각의 캐시가 특정 어드레싱 레지스터와 연계되게 하는 단계와,
 어드레싱 레지스터의 최상위 비트에서 기록(writing) 작업 후, 대응 캐시에서 캐시 로드 작업을 수행하는 단계를 포함하는 병렬 처리 방법.

청구항 16

제 15 항에 있어서,
 각각의 프로세서는 단일 스레드를 처리하도록 구성되고, 상기 방법은,
 단일 스레드를 처리하기 위해 복수의 프로세서 중 하나를 할당하는 단계와,
 상기 프로세서에 의해 각각의 할당된 스레드를 처리하는 단계와,
 상기 프로세서에 의해 처리되는 각각의 스레드로부터의 결과를 처리하는 단계와,
 스레드가 처리된 후 상기 복수의 프로세서 중 하나를 탈-할당하는 단계를 더 포함하는, 병렬 처리 방법.

청구항 17

삭제

청구항 18

삭제

청구항 19

삭제

청구항 20

삭제

청구항 21

삭제

청구항 22

삭제

청구항 23

삭제

청구항 24

삭제

명세서

기술분야

이 출원은 2006년 2월 3일자 US 가특허출원 제60/764,955호로부터 우선권을 주장한다. 상기 가출원의 전체 내용은 본원에서 참조로서 인용된다.

배경기술

[0001]

- [0002] 2가지 일반적인 접근법을 이용하여 컴퓨터 속도가 증가될 수 있는데, 인스트럭션 실행 속도를 증가시키는 것, 또는 병렬로 보다 많은 인스트럭션을 실행하는 것이 그것이다. 인스트럭션 실행 속도가 실리콘의 전자 이동 한계에 접근함에 따라, 병렬처리(parallelism)가 컴퓨터 속도의 증가의 가장 바람직한 대안이 되고 있다.
- [0003] 병렬처리에 대한 이전 시도들은 다음과 같다:
- [0004] 1. 현재 인스트럭션 실행과 다음번 인스트럭션 인출(instruction fetching)의 오버래핑(overlapping).
- [0005] 2. 인스트럭션 파이프라이닝(pipelining). 인스트럭션 파이프라이닝은 각각의 인스트럭션을 가능한 많은 조각으로 쪼개고, 순차적 인스트럭션(sequential instruction)을 병렬 실행 유닛으로 매핑(mapping)하기를 시도한다. 멀티-단계 인스트럭션의 비효율과, 소프트웨어 프로그램이 병렬 실행 유닛이 채워진 상태를 유지하기 위해 충분히 많은 순차적 인스트럭션을 제공할 수 없다는 것과, 브랜치(branch), 또는 루프, 또는 케이스 구조가 실행 유닛의 재-충전의 요구에 직면할 때 소비되는 큰 시간 페널티 때문에, 이론적인 최대 개선은 거의 얻어지지 않는다.
- [0006] 3. SIMD(Single Instruction Multiple Data). 이러한 종류의 기법은 Intel Pentium 3 및 그 밖의 다른 프로세서로 구현되는 것과 같은 Intel SSE 인스트럭션 세트에서 발견된다. 이러한 기법에서, 단일 인스트럭션(single instruction)이 다중 데이터(multiple data) 세트 상에서 실행된다. 이 기법은 비디오 그래픽 렌더링 등의 특수 애플리케이션에 대해서만 유용하다.
- [0007] 4. 하이퍼큐브(Hypercube). 이 기법은 큰 2차원 어레이를 이용하며, 때때로 프로세서와 로컬 메모리의 3차원 어레이를 이용한다. 이들 프로세서의 어레이를 지원하기에 필수적인 통신 및 상호연결이 이 기법을 매우 특수화된 애플리케이션으로 국한시킨다.
- [0008] 파이프라인은 하나의 인스트럭션의 실행(가령, 인출, 디코딩, 실행, 저장 등)의 조각을 연속적으로 수행하는 다수의 순차적 단계로 구성된 인스트럭션 실행 유닛(instruction execution unit)이다. 몇 개의 파이프라인이 병렬로 위치되어, 프로그램 인스트럭션이 차례로, 모든 파이프라인이 하나의 인스트럭션을 실행할 때까지, 각각의 파이프라인으로 유입될 수 있다. 그 후, 인스트럭션 채우기(instruction filling)는 본래의 파이프라인을 이용하여 반복된다. N개의 파이프라인이 인스트럭션으로 채워지고, 실행할 때, 성능 효과는 단일 실행 유닛에 대한 실행 속도의 N배 증가와 이론적으로 동일하다.
- [0009] 성공적인 파이프라이닝은 다음의 조건에 따라 좌우된다.
- [0010] 1. 하나의 인스트럭션의 실행이 몇 개의 연속적인 단계로서 정의될 수 있어야 한다.
- [0011] 2. 각각의 인스트럭션은 동일한 개수의 단계를 가져야 한다.
- [0012] 3. 인스트럭션 당 상태의 개수에 의해 병렬 실행 유닛의 최대 개수가 판단된다.
- [0013] 파이프라이닝이 병렬 파이프라인의 개수를 바탕으로 하는 성능 증가를 성취할 수 있기 때문에, 그리고 병렬 파이프라인의 개수는 하나의 인스트럭션의 단계의 개수에 의해 판단되기 때문에, 파이프라인은 복잡한 다중-단계 인스트럭션을 촉진한다.
- [0014] 과하게 파이프라이닝된 컴퓨터는 병렬 파이프라인 실행 유닛으로부터 기대되어지는 이론적 성능 향상만큼도 거의 성취하지 않는다. 이러한 파이프라인 페널티에 대한 몇 가지 이유로는 다음과 같다:
- [0015] 1. 소프트웨어 프로그램은 단지 순차적 인스트럭션만으로 이뤄지는 것이 아니다. 다양한 연구에 따라 실행 흐름의 변화가 매 8-10개의 인스트럭션마다 발생한다. 프로그램 흐름을 바꾸는 임의의 브랜치가 파이프라인을 혼란시킨다. 파이프라인 혼란을 최소화하기 위한 시도는 복잡하며, 불완전하게 완화시키는 경향이 있다.
- [0016] 2. 모든 인스트럭션이 동일한 개수의 단계를 갖도록 강제하는 것은 종종 최소 공통 분모의(즉, 가장 느리고 가장 복잡한) 인스트럭션을 갖는 실행 파이프라인을 초래한다. 파이프라인 때문에, 모든 인스트럭션이 동일한 개수의 단계로 강제된다. 상기 인스트럭션들이 상기 개수의 단계를 요구하느냐의 여부에 관계없이 말이다. 예를 들어, 논리 연산(가령, AND, 또는 OR)은 크기 순으로 ADD보다 더 빨리 실행되지만, 종종 이들 모두는 동일한 실행 시간을 할당받는다.
- [0017] 3. 파이프라인은 다중-단계 복합 인스트럭션을 촉진시킨다. 2개의 단계를 필요로 할 수 있는 인스트럭션은 20개의 단계를 채우기 위해 늘려지는데, 이는 파이프라인의 깊이(depth) 때문이다. (Intel Pentium 4는 20단계 파이프라인을 이용한다)

- [0018] 4. 각각의 파이프라인 단계에 대하여 요구되는 시간은 논리 회로 및 이에 연계된 트랜지스터를 통과하는 전파 딜레이와, 이에 추가로, 특정 단계에 대한 설계 오차에 해당한다.
- [0019] 5. 파이프라인 레지스터 및 그 밖의 다른 자원 액세스에 대한 중재는 종종, 중재 로직(arbitration logic)의 트랜지스터의 전파 딜레이로 인해 성능을 감소시킨다.
- [0020] 6. 추가적인 단계가 실행의 속도를 높이기보다는 실제로는 실행을 느리게 하기 전에, 인스트럭션이 쪼개질 수 있는 단계의 개수에는 상한이 존재한다. 일부 연구들에 따르면, Digital Equipment Corporation의 Alpha 프로세서의 마지막 세대의 파이프라인 구조(pipeline architecture)는 이러한 포인트를 초과했고, 실제로는 이전의 더 짧은 파이프라인의 프로세서 버전보다 더 느리게 수행됨을 알 수 있다.
- [0021] **파이프라인 쪼개기(Splitting Apart the Pipelines)**
- [0022] CPU 디자인의 리팩토링(re-factoring)에 대한 한 가지 양태는, 다수(N)의 단순화된 프로세서로 쪼개지는 파이프라이닝된 실행 유닛의 개념이다. (이러한 디자인에서 레지스터와 그 밖의 다른 일부 로직은 복제(duplicate)될 필요가 있다) N개의 단순화된 프로세서 각각은 앞서 언급된 파이프라이닝된 구조에 비해 다음의 장점을 갖는다.
- [0023] 1. 어떠한 파이프라인도 지연되지 않는다. 어떠한 브랜치 예측도 필요하지 않다.
- [0024] 2. 모든 인스트럭션이 가장 느린 인스트럭션과 동일한 실행 시간을 할당받는 것이 아니라, 인스트럭션들은 필요한 시간만큼을 취할 수 있다.
- [0025] 3. 필수적인 실행 단계로 축소시키고, 이로 인해서 파이프라인 패널티를 축소시킴으로써, 인스트럭션은 단순화될 수 있다.
- [0026] 4. 파이프라인으로부터 제거되는 각각의 단계에 의해, 전파 딜레이가 제거될 수 있고, 단계에 필요한 설계 오차가 제거될 수 있다.
- [0027] 5. 레지스터 중재(register arbitration)가 생략될 수 있다.
- [0028] 덧붙이자면, N개의 단순화된 프로세서를 갖는 시스템은 파이프라이닝된 CPU(pipelined CPU)에 비교할 때, 다음의 장점들을 가질 수 있다:
- [0029] 1. 최대 파이프라인 병렬처리의 한계가 없어질 것이다.
- [0030] 2. 파이프라이닝된 프로세서와 달리, 다중 자립형 프로세서(standalone processor)는 선택적으로 파워-다운(power down)되어, 사용되지 않을 때의 전력 소모를 줄일 수 있다.
- [0031] **병렬처리의 현재 접근법에 동반되는 그 밖의 다른 문제점들**
- [0032] 병렬처리의 다수의 구현예들이 Amdahl의 법칙을 극복하지 못한다. 병렬처리를 통한 가속은 문제점들 중 비-직렬화 부분으로 인한 오버헤드에 의해 제한된다. 필수적으로, 병렬처리의 양이 증가함에 따라서, 이를 지원하기 위해 필수적인 통신(communication)이 병렬처리로 인한 이득을 압도한다.
- [0033] **레드라인에 위치하는 정지등(Stoplight Sitting at Redline)**
- [0034] 현재의 프로세서의 또 다른 비효율은, 즉각적인 컴퓨팅 요구를 충족시키도록 컴퓨팅 파워를 확장할 수 없다는 것이다. 대부분의 컴퓨터는 무언가가 발생하기를 대기하면서 시간을 낭비한다. 컴퓨터들은 I/O, 다음 인스트럭션, 메모리 액세스, 또는 때때로 인간과의 인터페이스를 대기한다. 이러한 대기는 컴퓨팅 파워의 비효율적인 낭비이다. 덧붙이자면, 컴퓨터의 대기에 소모되는 시간은 종종, 증가된 파워 소모와 열 발생을 초래한다.
- [0035] 엔진 제어기, 신호 프로세서 및 방화벽 라우터 등의 애플리케이션은 대기 규칙의 예외이다. 이들 애플리케이션은 문제점 및 해결책 세트의 정의된 속성 때문에 병렬처리 가속의 바람직한 후보이다. N번의 독립적인 곱셈의 곱을 요구하는 문제는 N번 승수를 이용하여 더 빠르게 해결될 수 있다.
- [0036] 범용 컴퓨터의 인지 성능(perceived performance)은 자신의 피크 성능이다. 이에 근접하게 범용 컴퓨터가 바빠지는 것은 빠른 스크린 재생을 동반하는 비디오 게임 실행, 또는 커다란 소스 파일 컴파일하기, 또는 데이터베이스 검색하기이다. 최적의 환경에서, 비디오 렌더링은 특수 목적의 음영/변환/렌더링 하드웨어의 한 요소가 될

것이다. 프로그래밍을 이러한 특수 목적의 하드웨어의 한 요소로 하는 하나의 방법이 “스레드(thread)”의 사용이다.

- [0037] 스레드는, 자급식이면서 드물게 다른 스레드와 데이터를 통신하는 독립적인 프로그램이다. 스레드의 일반적인 용도는 느린 실시간 활동으로부터 데이터를 수집하고, 조합된 결과를 제공하는 것이다. 스레드는 또한 디스플레이 상의 변화를 렌더링하기 위해 사용될 수 있다. 스레드는 또 다른 스레드와의 추가적인 상호작용을 요청하기 전에, 수천, 또는 수백 개의 단계를 통해 변천할 수 있다. 독립적인 스레드는 병렬처리를 통한 증가된 성능에 대한 기회를 제공한다.
- [0038] 다수의 소프트웨어 컴파일러가, 소프트웨어 디자인 프로세스를 팩터링하기 위한 목적으로 스레드의 생성과 관리를 지원한다. 동일한 팩터링이, 바람직한 실시예의 스레드에 의해 최적화되는 마이크로프로세서(TOMI)에서의 스레드 레벨 병렬처리의 기법을 통해 다중 CPU 병렬 프로세싱을 지원할 것이다.
- [0039] **스레드 레벨 병렬처리(Thread Level Parallelism)**
- [0040] 스레딩은 소프트웨어 프로그램을 하나의 단일 CPU 상에 팩터링(factoring)하기 위한 잘 알려진 기법이다. 스레드 레벨 병렬처리는 TOMI 프로세서를 사용함으로써, 프로그램 가속을 얻을 수 있다.
- [0041] 다른 병렬처리 접근법에 비한 TOMI 프로세서의 한 가지 확실한 이점은 TOMI 프로세서가 현재 소프트웨어 프로그래밍 기법에 대한 최소의 변화를 요구한다는 것이다. 새로운 알고리즘은 개발될 필요가 없다. 많은 기존의 프로그램들이 재-컴파일(re-compile)될 필요는 있지만, 다시 쓰여질 필요는 없다.
- [0042] 효과적인 TOMI 컴퓨터 구조는 다수의 단순화된 프로세서 주변에서 구축되어야 한다. 여러 다른 타입의 컴퓨팅 문제에 대하여 여러 다른 구조가 사용될 수 있다.
- [0043] **기본 컴퓨터 연산(Fundamental Computer Operation)**
- [0044] 범용 컴퓨터에 있어서, 감소하는 주파수 수준의 가장 일반적인 연산은 로드 및 저장(Load and store)과, 시퀀싱(sequencing)과, 수리 및 논리(math and logic)이다.
- [0045] **로드 및 저장(Load and store)**
- [0046] LOAD 및 STORE의 매개변수는 출발지(source)와 도착지(destination)이다. LOAD 및 STORE의 파워는 출발지와 도착지의 범위이다(가령, 4Gbyte가 256byte보다 더 파워풀한 범위이다). 많은 데이터 세트에 있어서, 현재 출발지와 도착지에 대한 집약성(locality)이 중요하다. 플러스 1, 마이너스 1이 가장 유용하다. 현재의 출발지와 도착지로부터의 오프셋(offset)을 증가시킴으로써, 점진적으로 덜 유용해진다.
- [0047] LOAD 및 STORE는 또한 메모리 계층구조(memory hierarchy)에 의해 영향 받을 수 있다. 저장장치로부터의 LOAD는 CPU가 수행할 수 있는 가장 느린 연산일 수 있다.
- [0048] **시퀀싱(Sequencing)**
- [0049] 브랜치(branch)와 루프(loop)가 기본 시퀀싱 인스트럭션이다. 인스트럭션 시퀀스는 컴퓨터가 결정을 내리는 방식인 테스트를 기반으로 변한다.
- [0050] **수리 및 논리(Math and Logic)**
- [0051] 수리 및 논리 연산은 3가지 연산 중 가장 적게 사용된다. 논리 연산은 CPU가 수행할 수 있고, 단일 로직 게이트의 딜레이만큼 적은 딜레이를 필요로 할 수 있는 가장 빠른 연산이다. 상위 비트(higher order bit)가 하위 비트(lower order bit) 연산의 결과에 따라 좌우되기 때문에, 수리 연산은 더 복잡하다. 캐리 예측(carry lookahead)을 이용할 때조차, 32-비트 ADD는 32개 이상의 게이트 딜레이를 필요로 할 수 있다. 쉬프트 앤드 애드(shift and add) 기법을 이용하는 MULTIPLY는 32개의 ADD에 상응하는 것을 필요로 할 수 있다.
- [0052] **인스트럭션 크기의 상충(Tradeoffs of Instruction Size)**
- [0053] 완벽한 인스트럭션 세트는 무한개의 가능한 출발지, 도착지, 연산 및 다음 인스트럭션을 선택하기에 충분히 큰 op-코드로 구성될 것이다. 불행하게도, 완벽한 인스트럭션 세트의 op-코드는 무한으로 넓을 것이며, 따라서 인스트럭션 대역폭이 0이 될 것이다.
- [0054] 높은 인스트럭션 대역폭을 위한 컴퓨터 설계는, 가장 적은 수의 op-코드 비트를 갖고 가장 일반적인 출발지, 도착지, 연산 및 다음 인스트럭션을 효과적으로 정의할 수 있는 op-코드를 갖는 인스트럭션 세트의 생성을 포함한

다.

- [0055] 넓은 op-코드는 높은 인스트럭션 버스 대역폭을 요구하고, 최종 구조는, 컴퓨터의 성능이, 메모리로부터 인스트럭션이 인출되는 속도에 의해 제한되는 폰 노이만 병목현상(Von Neumann bottleneck)에 의해 빠르게 제한될 것이다.
- [0056] 메모리 버스가 64비트 폭을 갖는 경우, 각각의 메모리 사이클에서, 하나의 64-비트 인스트럭션, 또는 2개의 32-비트 인스트럭션, 또는 4개의 16-비트 인스트럭션, 또는 8개의 8-비트 인스트럭션을 인출할 수 있다. 32-비트 인스트럭션은 16-비트 인스트럭션의 2배만큼 유용한데, 왜냐하면 인스트럭션 대역폭이 절반이기 때문이다.
- [0057] 인스트럭션 세트 설계의 주요 목적은 인스트럭션 중복(redundancy)을 감소시키는 것이다. 일반적으로, 최적화된 효율적인 인스트럭션 세트는 인스트럭션과 데이터 모두의 집약성(locality)을 이용한다. 가장 쉬운 인스트럭션 최적화가 감망되어왔다. 대부분의 컴퓨터 프로그램에 있어서, 가장 가능성 높은 다음 인스트럭션은 메모리에서 순차적으로 다음에 위치하는 인스트럭션이다. 따라서 모든 다음 인스트럭션 필드를 갖는 인스트럭션 대신, 대부분의 인스트럭션은 다음 인스트럭션은 현재 인스트럭션 +1이라고 가정한다. 출발지에 대하여 0비트를, 도착지에 대하여 0비트를 갖는 구조를 생성하는 것이 가능하다.
- [0058] **스택 구조(Stack Architecture)**
- [0059] 또한 스택 구조 컴퓨터는 0 피연산자 구조(zero operand architecture)라고 일컬어진다. 스택 구조는 푸시 다운 스택(push down stack)의 내용물을 기반으로 하여 모든 연산을 수행한다. 2개의 피연산자 연산은 2개의 피연산자 모두 스택 상에 존재할 것을 필요로 한다. 연산이 실행될 때, 두 개의 피연산자 모두는 스택으로부터 팝(POP)되며, 연산이 실행되어, 결과가 스택 상으로 다시 푸시(PUSH)될 것이다. 출발지와 도착지가 스택 상에서 존재하는 것처럼 여겨질 수 있기 때문에, 스택 구조 컴퓨터는 매우 짧은 op-코드를 가질 수 있다.
- [0060] 대부분의 프로그램은 필요할 때마다 스택 상에서 이용가능한 것은 아닐 수 있는 전역 레지스터(global register)의 내용물을 필요로 한다. 이러한 사건을 최소화하기 위한 시도에는, 스택의 가장 위쪽에 존재하는 피연산자가 아닌 다른 피연산자를 액세스하는 것을 가능케 하는 스택 인덱싱(stack indexing)이 포함된다. 스택 인덱싱은, 스택 인덱스 값을 스택 자체에게 위치시키기 위한 더 큰 인스트럭션이나 추가적인 연산을 야기하는 추가적인 op-코드 비트를 필요로 한다. 종종 하나 이상의 추가적인 스택이 형성된다. 더 낫지만, 최적화된 것은 아닌 솔루션은 스택/레지스터 구조 조합이다.
- [0061] 또한 스택 구조 연산은 종종, 명백한 최적화에 반하는 방식으로 중복이다. 예를 들자면, 각각의 POP과 PUSH 연산은, 스택이 메모리에서 조작되기 때문에 시간을 낭비하는 메모리 연산을 초래할 가능성을 갖는다. 덧붙이자면, 스택 연산은 다음 연산에서 즉시 필요한 피연산자를 소모할 수 있으며, 이로 인해서, 또 다른 메모리 연산의 가능성을 갖는 피연산자 복제(operand duplication)가 필요할 수 있다. 1차원 어레이의 모든 요소에 15를 곱하는(multiply) 연산의 예를 들어보자.
- [0062] 스택 구조 상에서, 이 연산은 다음과 같다:
- [0063] 1. 어레이의 시작 주소를 푸시하기 (PUSH start address of array)
- [0064] 2. 주소를 복제하기(DUPLICATE address) (따라서 어레이의 결과를 저장하기 위한 주소를 갖는다)
- [0065] 3. 주소를 복제하기(DUPLICATE address) (따라서 어레이로부터 판독할 주소를 갖는다)
- [0066] 4. 간접 주소 푸시하기(PUSH INDIRECT) (스택의 가장 상위(top)에 의해 가르쳐지는(pointed) 어레이 위치의 내용을 푸시하기)
- [0067] 5. 15를 푸시하기(PUSH 15)
- [0068] 6. 곱셈(MULTIPLY) (라인 3.에서 읽은 어레이 내용을 15배하기)
- [0069] 7. 교환(SWAP) (다음 인스트럭션에 대하여 스택의 가장 상위 상의 어레이 주소를 얻기)
- [0070] 8. 간접 주소 팝하기(PUSH INDIRECT) (곱셈 결과를 POP하고, 어레이에 다시 저장한다)
- [0071] 9. 증연산(INCREMENT) (다음 어레이 아이템을 가리키기)
- [0072] 10. 어레이가 완료될 때까지 단계2.로 가기(GO TO)

- [0073] 라인 9.에서의 루프 카운터는 추가적인 매개변수를 필요로 할 것이다. 일부 구조에서, 이러한 매개변수는 또 다른 스택에 저장된다.
- [0074] 가설적 레지스터/어큐물레이터 구조에서의 예는 다음과 같이 구현될 수 있다:
- [0075] 1. 어레이 시작 주소의 포인터를 저장하기(STORE POINTER start address of array)
- [0076] 2. 포인터 읽기(READ POINTER) (가르쳐지는 주소의 내용을 어큐물레이터로 읽기)
- [0077] 3. 15를 곱하기(MULTIPLY 15)
- [0078] 4. 포인터 저장하기(STORE POINTER) (가르치고 있는 주소로 결과를 저장하기)
- [0079] 5. 포인터 증연산하기(INCREMENT POINTER)
- [0080] 6. 어레이가 완료될 때까지 라인 2.로 가기(GO TO)

[0081] 앞의 예에 대하여, 스택 구조의 9개의 단계와 레지스터 구조의 5개의 단계를 비교해 보자. 덧붙이자면, 스택 연산은 스택 연산으로 인한 추가적인 메모리 액세스에 대한 3번 이상의 기회를 갖는다. 가설적 레지스터/어큐물레이터 구조의 루프 제어는 레지스터에서 쉽게 다뤄질 수 있다.

[0082] 수식의 값을 구하기에는 스택이 유용하며, 대부분의 컴파일러에서 마찬가지로 사용된다. 또한 스택은 함수 호출 같은 중첩 연산에 대하여 유용하다. 대부분의 C 컴파일러는 스택을 이용하여 함수 호출을 구현한다. 그러나 범용 저장장치에 의한 보조 없이, 스택 구조는 다수의 추가적인 데이터 이동 및 조작을 필요로 한다. 최적화 목적을 위해, 또한 스택 PUSH 및 POP 연산이 수리 및 논리 연산으로부터 분리되어야 한다. 그러나 앞의 예에서 나타나듯이, 데이터를 반복적으로 로딩하고 저장할 때, PUSH INDIRECT 및 POP INDIRECT에 의해 어레이 주소가 소모되기 때문에, 스택은 특히 비효율적이다.

발명의 상세한 설명

[0083] 하나의 양태에서, 본 발명은 하나의 시스템을 포함하는데, 상기 시스템은 (a) 단일 칩 상의 다수의 병렬 프로세서와, (b) 상기 칩 상에 위치하고, 상기 프로세서 각각에 의해 액세스 가능한 컴퓨터 메모리를 포함하며, 이때 상기 프로세서 각각은 최소 허용되는(*de minimis*) 인스트럭션 세트를 프로세싱하며, 상기 프로세서 각각은 상기 프로세서의 3가지 이상의 특정 레지스터 각각에 대한 전용 로컬 캐시를 포함한다.

[0084] 변형 실시예에서, (1) 로컬 캐시 각각의 크기는 칩 상의 랜덤 액세스 메모리의 하나의 열에 상응하며, (2) 관련 캐시를 포함하는 3가지 이상의 특정 레지스터는 인스트럭션 레지스터, 출발지 레지스터 및 도착지 레지스터를 포함하고, (3) 최소 허용되는 인스트럭션 세트는 7개의 인스트럭션으로 구성되며, (4) 프로세서 각각은 하나의 단일 스레드를 프로세싱하며, (5) 어큐물레이터가 증연산 인스트럭션을 제외한 모든 인스트럭션에 대한 피연산자이고, (6) 각각의 인스트럭션에 대한 도착지는 항상 피연산자 레지스터이며, (7) 상기 3가지 레지스터는 자동-증연산하고, 상기 3가지 레지스터는 자동-감연산하며, (8) 각각의 인스트럭션은 완료되기 위해 단지 1회의 클럭 사이클만을 필요로 하며, (9) 상기 인스트럭션 세트는 어떠한 BRANCH 인스트럭션도 어떠한 JUMP 인스트럭션도 포함하지 않으며, (10) 각각의 인스트럭션은 8비트 이하의 길이를 가지며, (11) 하나의 단일 마스터 프로세서는 병렬 프로세서 각각을 관리하는 것을 담당한다.

[0085] 또 다른 양태에서, 본 발명은 하나의 시스템을 포함하는데, 상기 시스템은 (a) 하나의 단일 칩 상의 다수의 병렬 프로세서와, (b) 상기 칩 상에 위치하고, 상기 프로세서 각각에 의해 액세스 가능한 컴퓨터 메모리를 포함하며, 이때, 상기 프로세서 각각은 스레드(thread)-레벨의 병렬 프로세싱에 대하여 최적화된 인스트럭션 세트를 프로세싱한다.

[0086] 변형 실시예에서, (1) 상기 프로세서 각각은 최소 허용되는(*de minimis*) 인스트럭션 세트를 프로세싱하고, (2) 상기 프로세서 각각은 상기 프로세서의 3가지 이상의 특정 레지스터 각각에 대하여 전용인 로컬 캐시를 포함하며, (3) 상기 로컬 캐시 각각의 크기는 상기 칩 상의 랜덤 액세스 메모리의 하나의 행(row)에 상응하고, (4) 상기 3가지 이상의 특정 레지스터는 인스트럭션 레지스터, 출발지 레지스터 및 도착지 레지스터를 포함하고, (5) 상기 최소 허용되는(*de minimis*) 인스트럭션 세트는 7개의 인스트럭션으로 구성되며, (6) 상기 프로세서 각각은 하나의 단일 스레드(thread)를 프로세싱하며, (7) 하나의 단일 마스터 프로세서가 상기 병렬 프로세서 각각을 관리하는 것을 담당한다.

[0087] 또 다른 양태에서, 본 발명은 단일 칩 상에 위치하는 다수의 병렬 프로세서, 마스터 프로세서 및 컴퓨터 메모리를 이용하는 스레드-레벨 프로세싱 방법을 포함하며, 이때, 상기 다수의 프로세서 각각은 최소 허용되는(*de minimis*) 인스트럭션 세트를 프로세싱하고, 하나의 단일 스레드를 프로세싱하며, 상기 방법은 (a) 상기 다수의 프로세서 각각의 3가지 특정 레지스터 각각에게 로컬 캐시를 할당하는 단계와, (b) 하나의 단일 스레드를 프로세싱하기 위해, 다수의 프로세서 중 하나를 할당(allocating)하는 단계와, (c) 상기 프로세서에 의해, 각각의 할당된 스레드 프로세싱하는 단계와, (d) 상기 프로세서에 의해 프로세싱된 각각의 스레드로부터 결과를 프로세싱하는 단계와, (e) 스레드가 프로세싱된 후, 상기 다수의 프로세서 중 하나를 탈-할당(de-allocating)하는 단계를 포함한다.

[0088] 변형 실시예에서, (1) 상기 최소 허용되는(*de minimis*) 인스트럭션 세트는 7개의 인스트럭션으로 구성되며, (2) 상기 최소 허용되는(*de minimis*) 인스트럭션 세트의 각각의 인스트럭션은 8비트 이하의 길이를 가지며, (3) 상기 최소 허용되는(*de minimis*) 인스트럭션 세트의 각각의 인스트럭션은 1 클럭 사이클로 프로세싱된다.

실시예

[0103] 본 발명의 하나 이상의 실시예의 TOMI 구조는 바람직하게도 범용 컴퓨터로서 동작하기에 가능한 최소 로직을 사용한다. 가장 일반적인 연산은 우선순위를 부여받는다. 대부분의 연산은 컴파일러 최적화를 위해, 가시적(visible)이고, 규칙적이며, 이용가능하다.

[0104] 하나의 실시예에서, 도 1에서 도시되는 바와 같이, TOMI 구조는 어큐뮬레이터(accumulator), 레지스터 및 스택 구조의 변형이다. 이 실시예에서,

[0105] 1. 어큐뮬레이터 구조처럼, 어큐뮬레이터는 항상, 증연산 인스트럭션(increment instruction)을 제외하고, 피연산자 중 하나이다.

[0106] 2. 레지스터 구조처럼, 도착지는 항상 피연산자 레지스터 중 하나이다.

[0107] 3. 어큐뮬레이터 및 프로그램 카운터가 또한 레지스터 공간에 위치하며, 따라서 그 곳에서 연산될 수 있다.

[0108] 4. 3개의 특수 레지스터는 자동-증연산(auto-increment)되고, 자동-감연산(auto-decrement)되며, 입력과 출력에 대한 스택 및 스트림을 생성하기에 유용하다.

[0109] 5. 모든 연산은 단일 클럭 사이클(그리고 2개의 상태: 클럭 하이(clock high), 클럭 로우(clock low))을 필요로 한다.

[0110] 6. 모든 인스트럭션은 8비트 길이를 가져서, 인스트럭션 디코딩을 간단하게 하고, 속도를 높인다.

[0111] 7. 어떠한 BRANCH, 또는 JUMP 인스트럭션도 없다.

[0112] 8. 도 2에서 도시되는 바와 같이, 8비트 인스트럭션으로부터 3비트의 연산자 선택을 가능하게 하는 7개의 인스트럭션만 존재한다.

[0113] 바람직한 실시예의 장점 중 일부는 다음과 같다:

[0114] 1. 모든 연산은, 파이프라인에 의해 불가피한 등가로 제한되기보다는, 논리(logic)에 의해 허용되는 최대 속도에서 실행된다. 논리 연산은 가장 빠르다. 수리 연산은 그 다음 빠르다. 메모리 액세스를 필요로 하는 연산이 가장 느리다.

[0115] 2. 상기 구조는 임의의 데이터 폭으로 확장되고, 패키지 핀, 가산기 캐리 시간(adder carry time) 및 유용성(utility)에 의해서만 제한된다.

[0116] 3. 상기 구조는 범용 컴퓨터의 모든 연산을 수행하기에 필수적인 가능한 최소의 기능에 근접한다.

[0117] 4. 상기 구조는 매우 투명(transparent)하며, 매우 규칙적이고, 대부분의 연산은 최적화 컴파일러에 대하여 이용가능하다.

[0118] 상기 구조는 단일 모놀리식 칩(monolithic chip) 상에서 여러 번 복제되기에 충분히 단순하도록 설계된다. 하나의 실시예가 메모리를 모놀리식하게 포함하는 CPU의 다수의 복제본을 구현할 수 있다. 32-비트 CPU는 1500개보다 적은 게이트에서 구현될 수 있으며, 이때, 게이트의 대부분은 레지스터를 형성한다. 거의 1000개의 바람직한 실시예의 TOMI CPU가, 단일 Intel Pentium 4와 동일한 개수의 트랜지스터를 이용하여 구현될 수 있다.

[0119] 인스트럭션 세트

[0120] 인스트럭션 세트의 7개의 인스트럭션이, 그들의 비트 매핑(bit mapping)과 함께 도 2에서 도시된다. 각각의 인스트럭션은 하나의 8-비트 워드로 구성되는 것이 바람직하다.

[0121] 주소 지정 방식(Addressing mode)

[0122] 도 3은 서로 다른 주소 지정 방식의 유효 주소를 도시한다.

[0123] 주소 지정 방식은,

[0124] - 즉시(immediate) 주소 지정 방식

[0125] - 레지스터(register) 주소 지정 방식

[0126] - 레지스터 간접(register indirect) 주소 지정 방식

[0127] - 레지스터 간접 자동-증연산(register indirect auto-increment) 주소 지정 방식

[0128] - 레지스터 간접 자동-감연산(register indirect auto-decrement) 주소 지정 방식

[0129] 이다.

[0130] 특수 경우

[0131] 레지스터 0 및 레지스터 1 모두 프로그램 카운터(PC)를 참조한다. 피연산자로서 레지스터 0(PC)을 이용하는 모든 연산은, 어큐뮬레이터 캐리 비트(C)가 1일 때를 조건으로 한다. C=1인 경우, PC의 예전 값이 어큐뮬레이터(ACC)로 교환(SWAP)된다. 피연산자로서 레지스터 1(PC)을 이용하는 모든 연산은 조건이 없다.

[0132] 어떠한 브랜치도 존재하지 않음

[0133] 브랜치(Branch) 및 점프(Jump) 연산은 정확한 op-코드 공간의 다수의 비트를 필요로 하기 때문에, CPU 설계자에게 항상 문제가 된다. 브랜치 생성 함수는, LOADACC, xx를 이용하여 요망 브랜치 주소를 ACC로 로딩시킴으로써, 그 후, STOREACC, PC 인스트럭션을 이용함으로써, 생성될 수 있고, 브랜치를 발효(effect)할 수 있다. 레지스터 0을 저장함으로써, 브랜치는 C의 상태에 따라서 조건부로 만들어질 수 있다.

[0134] 스킵(Skip)

[0135] INC, PC를 실행함으로써, 스킵은 생성될 수 있다. 실행은 2번의 사이클을 필요로 할 것이다. 하나는 완성하기 위한 현재의 PC 증연산 사이클에 대한 것이며, 하나는 INC에 대한 것이다. 레지스터 0을 증연산함으로써, 스킵은 C의 상태에 따라 조건부로 만들어질 수 있다.

[0136] 상대 브랜치(relative branch)

[0137] 요망 오프셋(offset)을 ACC로 로딩시키고, ADD, PC 인스트럭션을 실행시킴으로써, 상대 브랜치가 생성될 수 있다. 레지스터 0에 더함으로써, 상대 브랜치는 C의 상태에 따라 조건부로 만들어질 수 있다.

[0138] 순방향 브랜치(forward branch)

[0139] 순방향 브랜치는 역방향 브랜치보다 더 유용한데, 왜냐하면, 루프의 탑(top)을 최초로 통과하는 프로그램 단계로서 PC를 저장함으로써, 루프에 있어서 필수인 역방향 브랜치의 위치는 쉽게 캡처되기 때문이다.

[0140] 상대 브랜치(relative branch)보다 효율적인 순방향 브랜치는, 브랜치 종점(endpoint)의 최하위 비트를 ACC로

로딩시키고, PC에 저장함으로써, 생성될 수 있다. 레지스터 0, 또는 레지스터 1의 사용에 따라서, PC는 조건부로, 또는 무조건부로 액세스될 수 있기 때문에, 순방향 브랜치 또한, 도착지 피연산자로서 PC 레지스터를 선택(레지스터 0, 또는 레지스터 1)하는 것에 의해, 조건부, 또는 무조건부일 수 있다.

[0141] 예를 들어,

[0142] LOAD1, #1C

[0143] STOREACC, PC

[0144] ACC의 최상위 비트가 0일 경우, 최상위 6비트만 PC 레지스터로 전송된다. 현재 PC 레지스터의 최하위 6비트가 로딩될 ACC 값보다 더 작은 경우, 레지스터의 최상위 비트는 변하지 않은 채 유지된다. 현재 PC 레지스터의 최상위 6비트가 로딩될 ACC 값보다 더 큰 경우, 현재 PC 레지스터가 증연산(increment)되어, 7번째 비트로 시작된다.

[0145] 이는 효과적으로, 31개 인스트럭션 앞까지의 브랜치를 가능하게 한다. 순방향 브랜치의 이러한 방법은 언제든지 사용될 수 있는데, 왜냐하면, 상대 브랜치의 3 인스트럭션에 비해 2 인스트럭션을 필요로 하는 것뿐 아니라, 가장 느린 연산 중 하나인 애더(adder)를 통과하는 것을 필요로 하지 않기 때문이다. 도 2A는 연산에서의 순방향 브랜치를 나타낸다.

[0146] **루프(loop)**

[0147] LOADACC, PC를 이용하여 루프의 탑(top)이 저장될 수 있다. 그 후, 루프 구조의 탑으로의 최종 포인터가 레지스터에 저장되거나, 오토인덱싱 레지스터(autoindexing register) 중 하나로 푸시(push)될 수 있다. 루프의 바텀(bottom)에서, 포인터는 LOADACC, EA를 이용하여 검색(retrieve)되고, STOREACC, PC를 이용하여 PC로 복귀될 수 있으며, 이로 인해서 역방향 루프(backward loop)가 도출된다. 레지스터 0으로 저장하고, 조건부 역방향 루프를 도출함으로써, 상기 루프는 C의 상태에 따라 조건부로 만들어질 수 있다.

[0148] **자기 수정 코드(self modifying code)**

[0149] STOREACC, PC를 이용하여 자기 수정 코드를 기록하는 것이 가능하다. 인스트럭션이 생성되거나, ACC로 인출되어, PC로 저장되어, 다음 인스트럭션으로서 실행될 수 있다. 이러한 기법은 CASE 구조를 생성하기 위해 사용될 수 있다.

[0150] 메모리의 점프 테이블 어레이가 N개의 주소와 JUMPTABLE의 기본 주소로 구성된다고 가정하자. 편의를 위해, JUMPTABLE은 낮은 메모리에 위치할 수 있고, 그 주소는 LOADI에 의해, 또는 LOADI와 이에 뒤따르는 하나 이상의 우향 쉬프트(right shift), ADD, ACC에 의해 생성될 수 있다.

[0151] 점프 테이블로의 인덱스가 ACC 내에 있고, 상기 점프 테이블의 기본 주소는 JUMPTABLE이라는 명칭의 범용 레지스터 내에 있다고 가정하자.

[0152] ADD, JUMPTABLE 점프 테이블의 기본 주소로의 인덱스를 더한다.

[0153] LOADACC, (JUMPTABLE) 인덱스된 주소를 로딩한다.

[0154] STOREACC, PC 점프를 실행한다.

[0155] 0000에서 시작하는 하위 메모리가 시스템 호출에 할당된 경우, 각각의 시스템 호출은 다음과 같이 실행될 수 있다(이때, SPECIAL_FUNCTION은 즉시 피연산자 0-63의 명칭임).

[0156] LOADI, SPECIAL_FUNCTION 시스템 호출 번호를 로딩한다.

[0157] LOADACC, (ACC) 시스템 호출의 주소를 로딩한다.

[0158] STOREACC, PC 함수로 점프한다.

[0159] **우향 쉬프트(right shift)**

[0160] 기본 구조는 우향 쉬프트 연산을 고려하지 않는다. 이러한 연산이 필요할 경우, 바람직한 실시예의 솔루션은 범용 레지스터 중 하나를 “우향 쉬프트 레지스터(right shift register)”로서 지정하는 것이다. STOREACC, RIGHTSHIFT는 ACC 우향 쉬프트된 단일 포지션을 “우향 쉬프트 레지스터”로 저장하며, 이때, 그 값은 LOADACC, RIGHTSHIFT라고 읽힐 수 있다.

[0161] 구조 확장성(Architectural Scalability)

[0162] TOMI 구조는 8-비트 인스트럭션을 특징으로 하는 것이 바람직하지만, 데이터 폭은 제한될 필요가 없다. 도 4는 임의의 4-32비트의 임의의 폭을 갖는 데이터 경로(data path)가 쉽게 생성되는 방법을 도시한다. 더 큰 폭의 데이터 핸들링을 생성하는 것은 레지스터 세트의 폭, 내부 데이터 경로 및 ALU의 폭을 요망 폭으로 증가시킬 것을 요구한다. 데이터 경로의 하한은 애더(adder)의 캐리 전파 딜레이 및 트랜지스터 예산에 의해서만 제한된다.

[0163] 바람직한 TOMI 구조는 단순성을 위해, 폰 노이만 메모리 구성(Von Neumann memory configuration)으로서 구현되지만, (별도의 데이터 및 인스트럭션 버스를 이용하는) 하버드 구조(Harvard architecture) 구현도 역시 가능하다.

[0164] 일반적인 수리 연산

[0165] 2의 보수 수식은 몇 가지 방식으로 해결될 수 있다. 범용 레지스터는 모두 “1”로서 사전에 구성될 수 있고, ALLONES라고 명명될 수 있다. 피연산자는 OPERAND라고 명명된다고 가정될 것이다.

[0166] LOADACC, ALLONES

[0167] XOR, OPERAND

[0168] INC, OPERAND OPERAND에서 2의 보수가 남겨진다.

[0169] 일반적인 컴파일러 구조

[0170] 대부분의 컴퓨터 프로그램은 컴파일러에 의해 발생된다. 따라서 유용한 컴퓨터 구조는 일반적인 컴파일러 구조에 적응되어야 한다.

[0171] 일반적으로 C 컴파일러는 매개변수를 함수 호출로 통과시키기 위한 스택을 유지할 것이다. S, 또는 X, 또는 Y 레지스터가 스택 포인터로서 사용될 수 있다. 함수 호출은, 예를 들어, STOREACC, (X)+를 이용하여, 매개변수를 스택으로서 기능하는 오토인덱싱 레지스터들 중 하나로 푸시(push)할 것이다. 함수가 시작되면, 매개변수는 사용되기 위해 범용 레지스터로 팝(POP)될 것이다.

[0172] 스택 상대 주소 지정(Stack Relative Addressing)

[0173] 범용 레지스터에서 딱 맞을 수 있는 것보다, 함수 호출로 통과되는 더 많은 요소가 존재하는 때가 있을 것이다. 다음의 예에서, 스택 푸시 연산이 스택을 감연산(decrement)한다고 가정된다. S가 스택 레지스터로서 사용되는 경우, 스택의 탑(top)에 관련 있는 N번째 아이템을 관독하기 위해,

[0174] LOADI, N

[0175] STOREACC, X

[0176] LOADACC, S

[0177] ADD, X

[0178] LOADACC, (X)

[0179] 어레이로의 인덱싱

- [0180] 어레이 함수로 입력이 있으면, 상기 어레이의 기본 주소가 ARRAY라고 명명된 범용 레지스터에 위치된다. 어레이에서 N번째 요소를 판독하기 위해,
- [0181] `LOADI, N`
- [0182] `STOREACC, X`
- [0183] `LOADACC, ARRAY`
- [0184] `ADD, X`
- [0185] `LOADACC, (X)`
- [0186] **N 워드 요소 어레이로의 인덱싱**
- [0187] 때때로 하나의 어레이가 요소의 N 워드 폭에 대하여 할당될 것이다. 어레이의 기본 주소는 ARRAY라고 명명된 범용 레지스터에 위치한다. 5 워드 폭 어레이에서 N번째 요소의 첫 번째 워드를 액세스하기 위해,
- [0188] `LOADI, N`
- [0189] `STOREACC, X` 임시 레지스터에 저장하기
- [0190] `ADD, ACC` 2를 곱하기
- [0191] `ADD, ACC` 다시 2를 곱하기(=4)
- [0192] `ADD, X` 1을 더하기(=5)
- [0193] `LOADACC, ARRAY`
- [0194] `ADD, X` 어레이의 기본 주소를 더하기
- [0195] `LOADACC, (X)`
- [0196] **로컬 TOMI 캐싱(Local TOMI Caching)**
- [0197] 캐시는 메인 메모리보다 더 작은 크기를 갖고, 더 빠른 액세스 시간을 갖는 메모리이다. 감소된 액세스 시간과 프로그램 및 데이터 액세스의 집약성에 의해, 캐시 연산이 많은 연산에 있어서 바람직한 TOMI 프로세서의 성능을 증가시킬 수 있다. 또 다른 관점에서, 메인 메모리에 대한 TOMI 프로세서의 독립성을 증가시킴으로써, 캐시는 병렬 프로세싱 성능을 향상시킨다. 메인 메모리에 대한 캐시의 상대적 성능과, 캐시로의(또는 캐시로부터의) 또 다른 메인 메모리 로드나 저장을 필요로 하기 전까지 TOMI 프로세서가 실행할 수 있는 사이클의 횟수가 TOMI 프로세서 병렬처리로 인한 성능 향상의 정도를 결정한다.
- [0198] TOMI 로컬 캐시가 TOMI 프로세서 병렬처리로 인한 성능 향상을 보장한다. 각각의 TOMI 프로세서는 도 5에서 도시되는 바와 같은 3개의 연계된 로컬 캐시를 갖는 것이 바람직하다, 즉 다음과 같다:
- [0199] 인스트럭션 - PC와 연계
- [0200] 출발지 - X 레지스터와 연계
- [0201] 도착지 - Y 레지스터와 연계
- [0202] 이들 캐시의 최적의 크기는 경우에 따라 다르다. 통상적인 실시예는 각각의 캐시에 대하여 1024 바이트를 필요로 할 수 있다. 다시 말하자면, 출발지와 도착지에 대하여 1024개의 인스트럭션 및 256개의 32-비트 워드가 필요하다. 둘 이상의 요인이 캐시의 최적 크기를 결정한다. 첫 번째는 또 다른 캐시 로드, 또는 저장 연산이 필요해지기 전까지의, TOMI 프로세서가 순환(cycle)할 수 있는 상태의 개수이다. 두 번째는 메인 메모리 연산 동안 가능한 TOMI 프로세서 실행 사이클의 횟수에 대한 메인 메모리로부터의 캐시 로드, 또는 저장 연산의 비용(cost)이다.

[0203] RAM에 TOMI 프로세서 내장하기(Embedding TOMI Processor in RAM)

[0204] 하나의 실시예에서, 광역 버스(wide bus)가 큰 내장형 메모리를 캐시에 연결시키고, 이로 인해서, 캐시로의 로드나 저장 연산이 빠르게 발생할 수 있다. RAM에 내장된 TOMI 프로세서를 이용하여, 전체 캐시의 로드, 또는 저장은 하나의 RAM 열(column)로의 단일 메모리 사이클로 구성될 것이다. 하나의 실시예에서, 내장형 메모리는 63개의 TOMI 프로세서의 요청에 응답할 것이며, 이로 인해서 하나의 TOMI 프로세서에 대한 캐시 로드, 또는 저장 응답 시간은, 또 다른 TOMI 프로세서 캐시의 로드, 또는 저장이 완료되는 동안, 연장될 수 있다.

[0205] 도 6에서 도시되는 바와 같이, 연계된 메모리 주소 지정 레지스터 X, Y, PC의 변화를 바탕으로 하여, 캐시는 저장되고 로딩될 수 있다. 예를 들어, PC 레지스터의 총 폭은 24비트일 수 있다. PC 캐시가 1024 바이트인 경우, PC의 하위 10비트가 PC 캐시 내에서의 액세스를 정의할 것이다. 상위 14비트에 변화가 있도록 PC가 기록될 때, 캐시 로드 사이클이 요구될 것이다. 캐시 로드 사이클이 완료되고, 지시된 인스트럭션이 PC 캐시로부터 인출될 수 있을 때까지, PC 캐시와 연계되는 TOMI CPU가 실행을 중단시킬 것이다.

[0206] 캐시 이중 버퍼링(Cache Double Buffering)

[0207] 보조 캐시(secondary cache)가 캐시 로드 요구치를 예상하여 로딩될 수 있다. 2개의 캐시는 동일하고, 대안적으로 PC의 상위 14비트의 내용을 기반으로 하여 선택되거나, 선택해제될 것이다. 앞의 예에서, PC의 상위 14비트가 보조 캐시에 미리 캐싱된 데이터의 비트와 일치하도록 변화되었을 때, 보조 캐시는 주 캐시(primary cache)로서 선택될 것이다. 이제 오래된 주 캐시는 보조 캐시가 될 것이다. 대부분의 컴퓨터 프로그램은 선형으로 메모리를 증가시키기 때문에, 본 발명의 하나의 실시예는, 현재 PC 더하기 1의 상위 14비트에 의해 참조되는 주 메모리의 캐시 내용의 내용을 항상 인출하는 보조 캐시를 가질 것이다.

[0208] 보조 캐시의 추가에 의해, 현재 캐시의 경계 밖으로 이동할 때, TOMI 프로세서가 메인 메모리로부터 인출될 메모리 데이터에 대하여 대기해야할 시간이 감소될 것이다. 보조 캐시의 추가는 TOMI 프로세서의 복잡도를 거의 2배로 만들 것이다. 최적의 시스템에 있어서, 이러한 복잡도를 2배로 만드는 것은 TOMI 프로세서의 성능을 2배로 만드는 것에 의해 상쇄되어야 한다. 그렇지 않다면, 보조 캐시 없는 2개의 더 단순한 TOMI 프로세서가 동일한 트랜지스터 카운트를 갖고 구현될 수 있다.

[0209] 고속 곱셈, 부동 소수점 연산, 추가적인 기능(High Speed Multiply, Floating Point Operations, Additional Functionality)

[0210] 정수 곱셈 및 모든 부동 소수점 연산은 많은 사이클이 수행될 것을 요구하며, 심지어 특수 목적의 하드웨어를 요구한다. 따라서 이들은 기본 TOMI 프로세서에 포함되기 보다는 다른 프로세서에 팩터링(factoring)되어야 한다. 디지털 신호 프로세싱(DSP) 연산은 종종, 총 곱셈이 다수의 사이클을 필요로 할 수 있을지라도, 매 사이클마다 결과를 생성하는 깊게 파이프라이닝된 멀티플라이어를 이용한다. 신호 프로세싱에 있어서, 동일한 알고리즘을 계속 반복하는 애플리케이션, 가령, 멀티플라이어 구조가 최적이며, 주변 프로세서(peripheral processor)로서 TOMI 프로세서와 일체될 수 있지만, TOMI 프로세서와 직접 일체된 경우, 복잡도를 증가시키고, 전체 성능을 감소시킬 가능성을 갖는다. 도 7은 광역 시스템 RAM 버스를 이용하도록 구성되는 추가적인 프로세싱 기능의 한 가지 예를 도시한다.

[0211] TOMI 인터럽트 전략

[0212] 인터럽트는 프로세스의 보통의 순차 연산의 외부에서 발생하는 이벤트이며, 이 이벤트에 의해, 프로세스는 자신의 연산의 순서를 즉시 변경한다.

[0213] 인터럽트의 예로는 외부 장치에 의한 연산의 완료, 또는 일부 하드웨어에 의한 오류 상태가 있다. 종래의 프로세서는 보통의 순차 연산을 빨리 중단시키고, 현재 연산의 상태를 저장하며, 인터럽트를 발생시킨 이벤트가 무엇이든지 이를 다루기 위한 일부 특수 연산을 수행하고, 상기 특수 연산이 완료될 때, 이전 상태를 복구시키고, 순차 연산을 계속하려 할 것이다. 인터럽트 핸들링의 질의 주요한 측정치는 응답 시간이다.

[0214] 인터럽트는 전통적인 프로세서에 대한 몇 가지 문제를 보여준다. 인터럽트는 실행 시간을 불명확하게 만든다. 인터럽트는 상태를 저장하고, 그 후 복구하는 프로세서 사이클을 낭비한다. 인터럽트는 프로세서 디자인을 복잡

하게 하며, 모든 프로세서 연산을 느리게 하는 딜레이를 발생시킬 수 있다.

[0215] 오류 핸들링의 경우와 프로세서가 실세계 활동과 직접 인터페이싱하는 경우를 제외하고, 즉시 인터럽트 반응은 대부분의 프로세서에게 불필요하다.

[0216] 멀티프로세서 TOMI 시스템의 하나의 실시예에서, 단지 하나의 프로세서만이 주 인터럽트 기능을 갖는다. 그 밖의 다른 모든 프로세서는 일부 할당된 작업을 완료하고, 그들 스스로 중단할 때까지, 또는 조정 프로세서(coordinating processor)에 의해 중단될 때까지, 인터럽트되지 않고 실행된다.

[0217] 입력/출력(I/O)

[0218] TOMI 프로세서 환경의 하나의 실시예에서, 단일 프로세서가 외부 세계(external world)로의 모든 인터페이싱을 담당한다.

[0219] 직접 메모리 액세스(DMA) 컨트롤

[0220] 하나의 실시예에서, TOMI 프로세서 시스템에서의 외부 세계에 대한 즉시 응답은 DMA 컨트롤러를 통해 발생한다. DMA 컨트롤러는, 외부 장치에 의해 요청될 때, 데이터를 외부 장치로부터, 시스템 RAM으로 기록하기 위해 내부 데이터 버스로 전송한다. 또한 동일한 컨트롤러가, 요청받을 때, 데이터를 시스템 RAM으로부터 외부 장치로 전송한다. DMA 요청이 내부 버스 액세스에 대한 가장 높은 우선순위를 가질 것이다.

[0221] TOMI 프로세서의 어레이를 구성하기

[0222] 본 발명의 바람직한 실시예의 TOMI 프로세서는 복수 개로 복제되고, 모놀리식 칩(monolithic chip) 상에서 추가적인 프로세싱 기능, 초 광역 내부 버스, 시스템 메모리와 조합되도록 설계된다. 이러한 시스템을 위한 외부 메모리 맵이 도 8에서 도시된다.

[0223] 각각의 프로세서에 대한 메모리 맵은 처음 32개의 로케이션(1F hex)을 상기 각각의 프로세서를 위한 로컬 레지스터 전용으로 둔다(도 3 참조). 메모리 맵의 나머지 부분은 모든 프로세서에 의해, 그들의 캐시 레지스터를 통해 주소 지정이 가능하다(도 6 참조). 시스템 RAM의 주소 지정 능력(addressability)은, 로컬 캐시와 연계되는 3개의 레지스터(PC, X 및 Y)의 폭에 의해서만 제한된다. 레지스터가 24 비트 폭을 갖는 경우, 총 주소 지정 능력은 4Mbyte일 것이지만, 상한은 없다.

[0224] 하나의 실시예에서, 64개의 TOMI 프로세서는 메모리와 모놀리식하게 구현된다. 하나의 단일 마스터 프로세서(master processor)가 나머지 63개의 프로세서를 관리를 담당한다. 슬레이브 프로세서 중 하나가 아이들(idle) 상태일 때, 클러킹(clocking)하지 않고, 따라서 전력을 거의, 또는 전혀 사용하지 않으며, 열을 거의, 또는 전혀 발생시키지 않는다. 초기화 시, 단지 마스터 프로세서만 동작한다. 스레드가 시작되어야 할 때까지 마스터는 인스트럭션을 인출하고 실행하기를 시작한다. 각각의 스레드는 선-컴파일(pre-compile)되어, 메모리로 로딩되었다. 스레드를 시작하기 위해, 상기 마스터는 스레드를 TOMI CPU 중 하나에게 할당한다.

[0225] 프로세서 이용가능성(processor availability)

[0226] 바람직하게 작업하기 위한 TOMI 프로세서의 이용가능성의 조정(coordination)은 도 9에서 도시된 프로세서 이용가능성 표(processor availability table)에 의해 다뤄진다. 조정(마스터) 프로세서는 다음의 기능을 수행할 수 있는 것이 바람직하다:

[0227] 1. 슬레이브 프로세서에 대한 호출 매개변수(가령, 스레드, 출발지 메모리 및 도착지 메모리의 실행 주소)를 자신의 스택에 푸시(push)하기

[0228] 2. 슬레이브 프로세서 시작하기

[0229] 3. 폴링(polling)에 의해, 또는 인터럽트에 응답하기에 의해, 슬레이브 프로세서 스레드 완료 이벤트에 응답하기

[0230] 프로세서 요청하기(Requesting a Processor)

[0231] 조정 프로세서가 유효성 표로부터 상기 프로세서를 요청할 수 있다. "0"으로 설정된 available_flag를 갖는 가장 하위 프로세서의 번호가 복귀(return)된다. 그 후, 조정 프로세서가 이용가능한 프로세서와 연계된 available_flag를 "1"로 설정하여, 슬레이브 프로세서를 시작할 수 있다. 어떠한 프로세서도 이용가능하지 않는 경우, 요청은 오류(error)를 복귀시킬 것이다. 대안적으로, 수행될 것이 요청된 작업과 관련된 우선순위 레벨을 기반으로 하는 조정 프로세서에 의해, 프로세서가 할당될 수 있다. 우선순위 체계를 기반으로 자원을 할당하기 위한 기법은 종래 기술에서 잘 알려져 있다. 도 10은 프로세서 할당의 3가지 바람직한 구성요소를 도시한다: 조정 프로세서 개시 연산, 슬레이브 프로세서 연산 및 인터럽트 응답을 통한 조정 프로세서 결과 핸들링이 그것이다.

[0232] 슬레이브 프로세서 단계별 시작하기(Step-by-Step Starting a Slave Processor)

[0233] 1. 조정 프로세서가 실행할 스레드에 대한 매개변수를 자신의 스택으로 푸시(push)한다. 매개변수는, 스레드의 시작 주소, 스레드에 대한 출발지 메모리, 스레드에 대한 도착지 메모리 및 마지막 parameter_count를 포함할 수 있다.

[0234] 2. 조정 프로세서는 이용가능한 프로세서를 요청한다.

[0235] 3. 프로세서 할당 논리(processor allocation logic)가, 설정된 관련 available_flag 및 빈(cleared) 관련 done_flag 모두를 갖는 숫자상으로 가장 낮은 슬레이브 프로세서의 번호를 복귀시키거나, 오류(error)를 복귀시킨다.

[0236] 4. 오류가 복귀된 경우, 조정 프로세서는, 슬레이브 프로세서가 이용가능해질 때까지 요청을 다시 시도하거나, 오류를 핸들링하기 위한 특수 연산을 수행할 수 있다.

[0237] 5. 이용가능한 프로세서 번호가 복귀되는 경우, 상기 조정 프로세서는 지시되는 프로세서에 대한 available_flag를 비운다. 이 연산은 스택 매개변수에 대한 parameter_count 번호를 선택된 슬레이브 프로세서의 스택으로 푸시(push)한다. done_flag가 0으로 비워진다.

[0238] 6. 슬레이브 프로세서가 스택의 탑(top)에 위치하는 아이템을 검색하고, 이를 슬레이브 프로세서의 프로그램 카운터에게 전송한다.

[0239] 7. 그 후, 상기 슬레이브 프로세서는 프로그램 카운터에 의해 지시되는 메모리 열(column)을 인스트럭션 캐시로 인출한다.

[0240] 8. 슬레이브 프로세서는 인스트럭션 캐시의 시작 부분에서부터 인스트럭션을 실행하기 시작한다. 첫 번째 인스트럭션이 스택으로부터 호출 매개변수를 검색할 가능성이 높을 것이다.

[0241] 9. 슬레이브 프로세서가 인스트럭션 캐시로부터 스레드를 실행시킨다. 스레드가 완료될 때, 관련 done_flag의 상태를 체크한다. 상기 done_flag가 설정된 경우, done_flag가 비워져서, 조정 프로세서가 임의의 이점 결과들을 핸들링했다는 것을 나타낼 때까지 대기한다.

[0242] 10. 슬레이브 프로세서와 관련된 인터럽트 벡터가 -1로 설정된 경우, done_flag를 설정함으로써 어떠한 인터럽트도 생성되지 않을 것이다. 따라서 조정 프로세서는 done_flag가 설정되도록 폴링(polling)할 수 있다.

[0243] 조정 프로세서가 done_flag가 설정된 사실을 검출할 때, 슬레이브 프로세서의 결과를 핸들링하고, 가능하면 슬레이브 프로세서에게 새로운 작업을 할 것을 재할당할 수 있다. 슬레이브 프로세서의 결과가 처리되었을 때, 관련 조정 프로세서가 관련 done_flag를 비울 것이다.

[0244] 슬레이브 프로세서와 관련되는 인터럽트 벡터가 -1이 아닌 경우, 관련 done_flag를 설정함으로써, 조정 프로세서가 인터럽트되어, 인터럽트 벡터 주소에서 인터럽트 핸들러(interrupt handler)를 실행시키기 시작할 것이다.

[0245] 관련 available_flag가 또한 설정되었는 경우, 조정 프로세서는 또한 슬레이브 프로세서의 스택 상으로 푸시(push)된 복귀 매개변수(return parameter)를 판독할 수 있다.

[0246] 상기 인터럽트 핸들러는 슬레이브 프로세서의 결과를 핸들링하며, 가능하다면, 슬레이브 프로세서에게 새로운

작업을 수행할 것을 재할당할 것이다. 슬레이브 프로세서의 결과가 처리되었을 때, 조정 프로세서 상에서 실행되는 인터럽트 핸들러가 관련 done_flag를 비울 것이다.

[0247] 11. done_flag가 비워져 있는 경우, 슬레이브 프로세서가 자신의 관련 done_flag를 설정하고, 새로운 start_time을 저장한다. 이러한 슬레이브 프로세서는 계속 작업하거나, 이용가능한 상태(available state)로 복귀될 수 있다. 이용가능한 상태로 복귀하기 위해, 슬레이브 프로세서는 복귀 매개변수를 자신의 스택과, 뒤이어 스택 카운트로 푸시(push)하고, 자신의 available_flag를 설정한다.

[0248] 메모리 잠금(Memory Locking)

[0249] TOMI 프로세서가 자신의 캐시를 통해 시스템 메모리를 판독하고 기록한다. 완전히 캐싱된 열(column)은 한 번에 판독, 또는 기록된다. 임의의 프로세서가 시스템 메모리의 임의의 부분을 판독할 수 있다. 개별적 프로세서는 자신이 배타적(exclusive)으로 기록하기 위해 메모리의 열을 잠글 수 있다. 이러한 잠금 메커니즘에 의해, 프로세서 간 메모리 기록 충돌이 피해진다.

[0250] 제안되는 적용예

[0251] 병렬처리는 개별적인 프로세서에 대한 작업의 독립적인 조각으로 팩터링될 수 있는 적용예를 효과적으로 가속한다. 바람직하게 팩터링하는 한 가지 적용예는 로봇 시각(robotic vision)을 위한 이미지 조작이다. 이미지 조작 알고리즘은 상관(correlation), 등화(equalization), 모서리 식별(edge identification) 및 그 밖의 다른 연산을 포함한다. 많은 것들이 행렬 조작(matrix manipulation)에 의해 수행된다. 도 11에서 도시되는 바와 같이, 상기 알고리즘은 바람직하게 팩터링한다.

[0252] 도 11에서, 예시적인 이미지맵이 24개의 프로세서를 보여주며, 이때, 프로세서는 총 이미지맵의 사각형 서브셋을 위한 이미지 데이터를 조작하도록 할당된다.

[0253] 도 12는 하나의 실시예에서, TOMI 시스템 RAM이 할당될 수 있는 방법을 도시한다. 시스템 RAM의 하나의 블록은 이미지 캡처 픽셀을 보유하며, 또 다른 블록은 처리된 결과를 보유한다.

[0254] 연산 동안, 조정 프로세서가 지정된 시간 마다 이미지 픽셀을 외부 소스로부터 내부 시스템 RAM으로 전송하기 위해 DMA 채널을 할당했다. 이미지 캡처의 통상적인 속도는 초 당 60개의 이미지일 수 있다.

[0255] 그 후, X 레지스터에 의해 사용될 이미지맵의 주소, Y 레지스터에 의해 사용될 처리된 이미지의 주소, 2의 매개변수 카운트, 이미지 프로세싱 알고리즘의 주소를 푸시(push)함으로써, 조정 프로세서가 슬레이브 프로세서 1을 활성화한다. 조정 프로세서는 프로세서 2 내지 25를 순차적으로, 그리고 유사하게 활성화한다. 이미지 프로세싱 알고리즘이 완료될 때까지, 프로세서는 계속하여 독립적으로, 그리고 병렬로 실행된다.

[0256] 알고리즘이 완료될 때, 각각의 프로세서가 프로세서 이용가능성 표(Processor Availability Table)에서 자신의 관련 done_flag를 설정한다. 조정 프로세서에 의해 결과가 핸들링되며, 이는 완료 또는, “종료(done)” 이벤트 후의 인터럽트에 대하여 응답할 것을 폴링하는 것이다.

[0257] 도 13은 64개의 프로세서의 모놀리식 어레이에 대한 예시적 평면도이다.

도면의 간단한 설명

[0089] 도 1은 하나의 실시예의 바람직한 TOMI 구조를 도시한다.

[0090] 도 2는 바람직한 인스트럭션 세트를 도시한다.

[0091] 도 2A는 연산 중의 순방향 브랜치를 도시한다.

[0092] 도 3은 여러 다른 주소 지정 방식의 여러 다른 유효 주소를 도시한다.

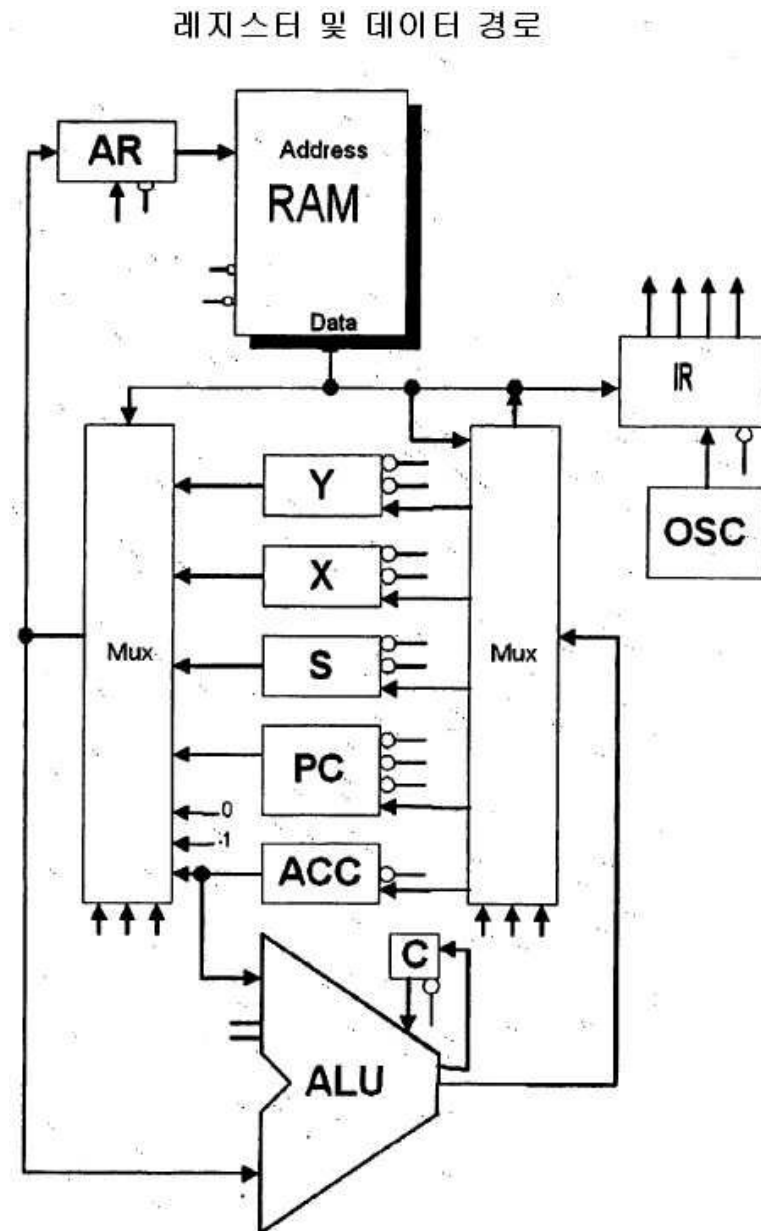
[0093] 도 4는 4-32 비트로부터 데이터 경로가 쉽게 생성되는 법을 도시한다.

[0094] 도 5는 예시적 로컬 캐시를 도시한다.

- [0095] 도 6은 예시적 캐시 관리 상태를 도시한다.
- [0096] 도 7은 광역 시스템 RAM 버스를 이용하기 위해 구성된 추가적인 프로세싱 기능의 하나의 실시예를 도시한다.
- [0097] 도 8은 예시적 메모리 맵을 도시한다.
- [0098] 도 9는 예시적 프로세스 이용가능성 표를 도시한다.
- [0099] 도 10은 프로세서 할당의 3가지 구성요소를 도시한다.
- [0100] 도 11은 바람직한 팩터링을 도시한다.
- [0101] 도 12는 바람직한 시스템 RAM을 도시한다.
- [0102] 도 13은 64개의 프로세서의 모놀리식 어레이에 대한 예시적 평면도를 도시한다.

도면

도면1



도면2

인스트럭션 Op 코드 맵

Op 코드	연산 부호	연산	캐리 이펙트
00x xxxxx	LOADI, X	X -> ACC	If zero, C=1
010 yyyyy	LOADACC, Y	EAY -> ACC	If zero, C=1
011 yyyyy	STOREACC, Y	ACC -> EAY	No change
100 yyyyy	ADD, Y	ACC + EAY -> EAY	C = carry
101 yyyyy	AND, Y	ACC AND EAY -> EAY	If zero, C=1
110 yyyyy	XOR, Y	ACC XOR EAY -> EAY	If zero, C=1
111 yyyyy	INC, Y	EAY + 1 -> EAY	If zero, C=1

x- binary immediate value

X - HEX immediate value

y - binary value of operand

Y - HEX value of operand

EAY - Effective address indicated by Y

도면2a

순방향 브랜치
(Forward Branches)

	MSB		LSB
PC	XXXXXXXX	XXXXXXXX	XXXXXXXXXX
ACC	00000000	00000000	00ZZZZZZ

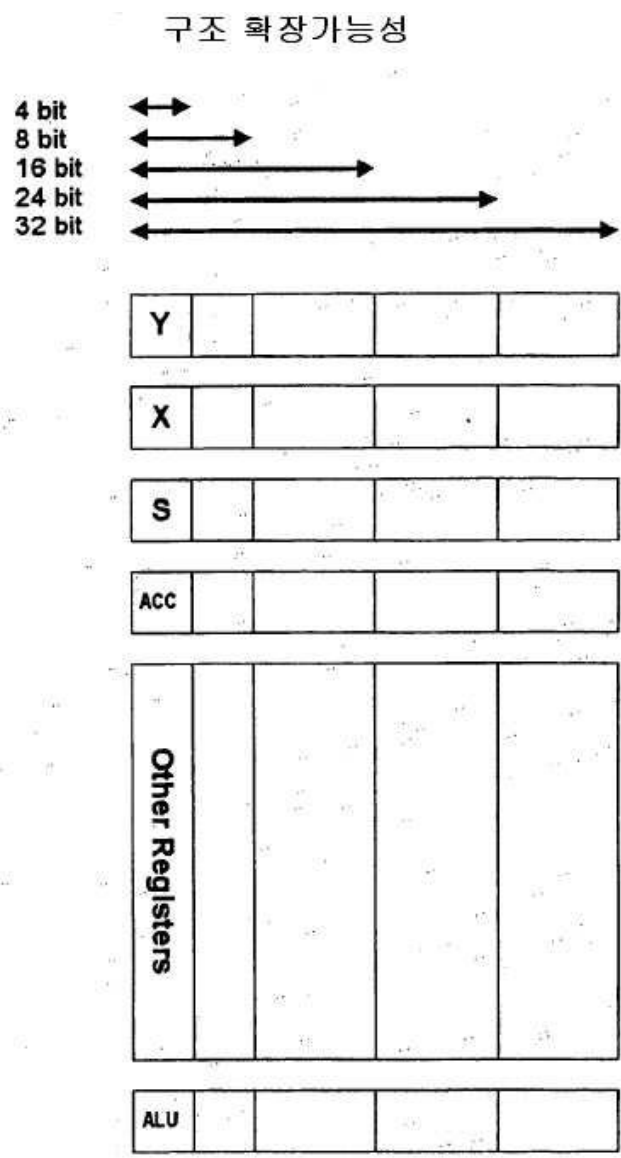
순방향 브랜치가 취해짐,  → 

if `wwwwwww > zzzzzz`,
then `xxxxxxxx xxxxxxxx xx + 1` → `xxxxxxxx xxxxxx xx`

도면3

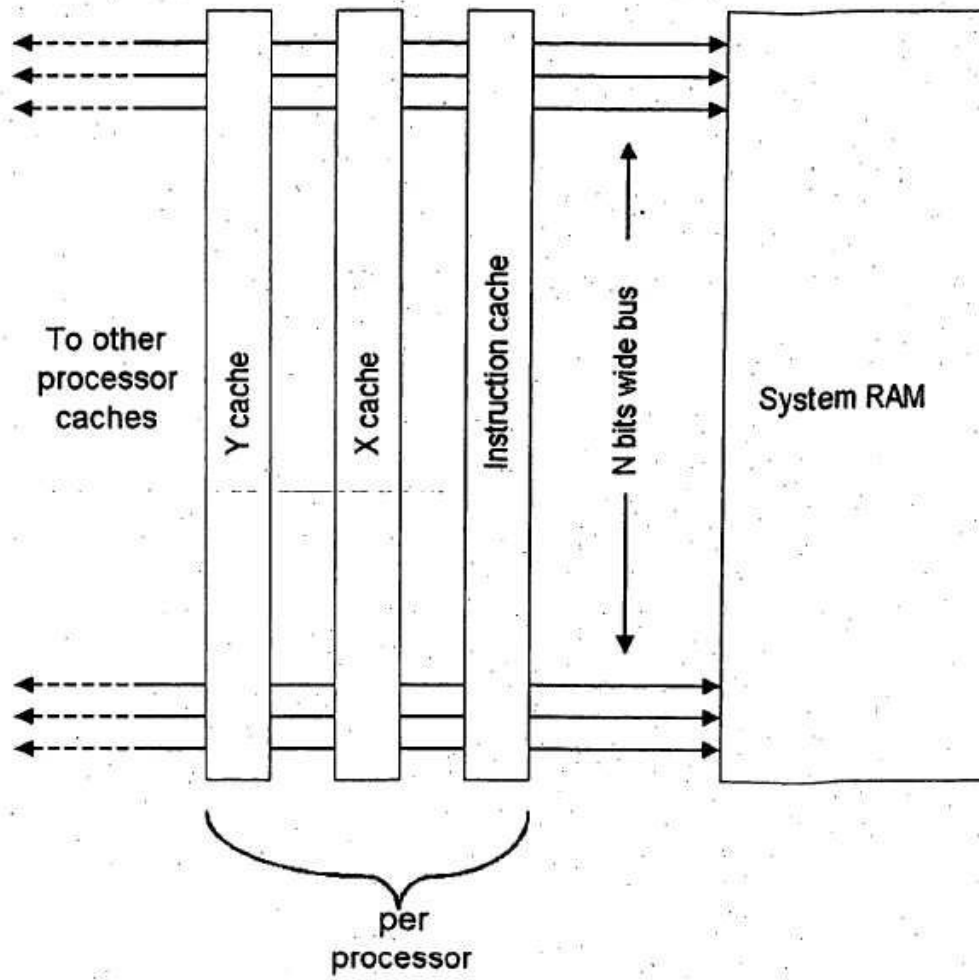
레지스터	유효 주소	특수 연산
00000	PC	All operations conditional on C=1
00001	PC	
00010	ACC	
00011	(ACC)	
00100	S	
00101	(S)	
00110	(S) 자동 증연산	
00111	(S) 자동 감연산	
01000	X	
01001	(X)	
01010	(X) 자동 증연산	
01011	(X) 자동 감연산	
01100	Y	
01101	(Y)	
01110	(Y) 자동 증연산	
01111	(Y) 자동 감연산	
10000	Register 10	
10001	Register 11	
10010	Register 12	
10011	Register 13	
10100	Register 14	
10101	Register 15	
10110	Register 16	
10111	Register 17	
11000	Register 18	
11001	Register 19	
11010	Register 1A	
11011	Register 1B	
11100	Register 1C	
11101	Register 1D	
11110	Register 1E	
11111	Register 1F	

도면4



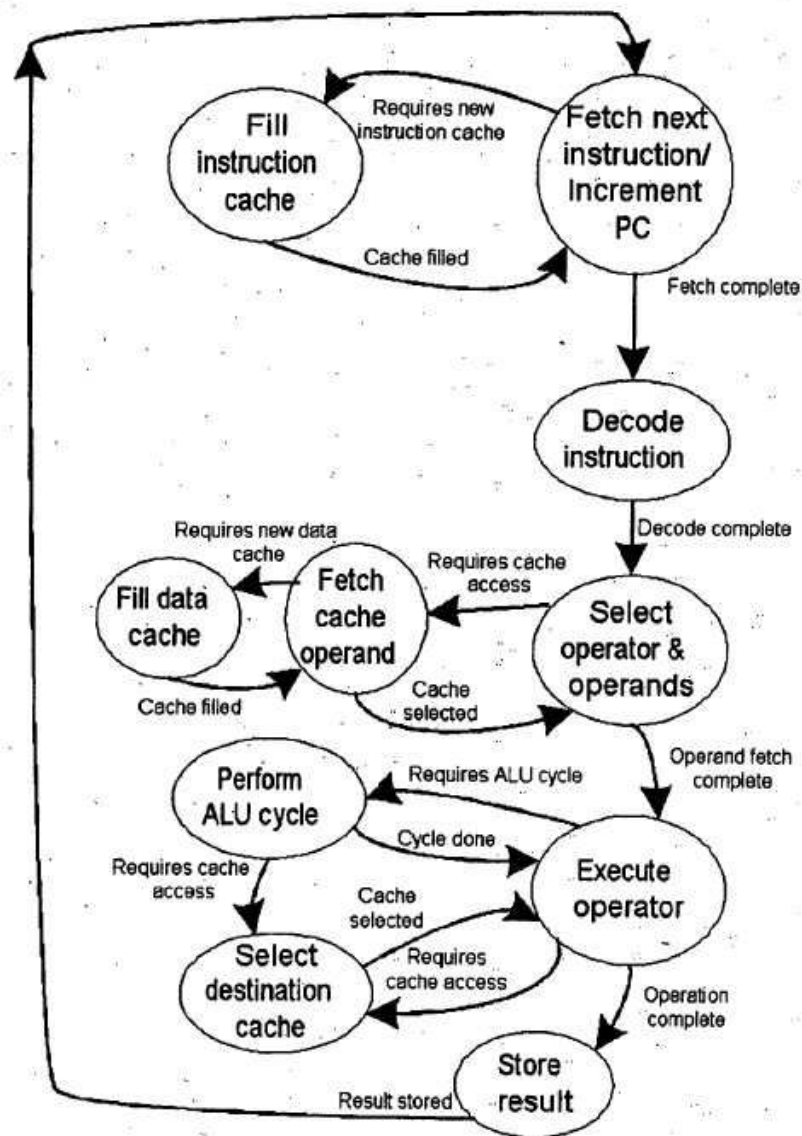
도면5

로컬 캐시



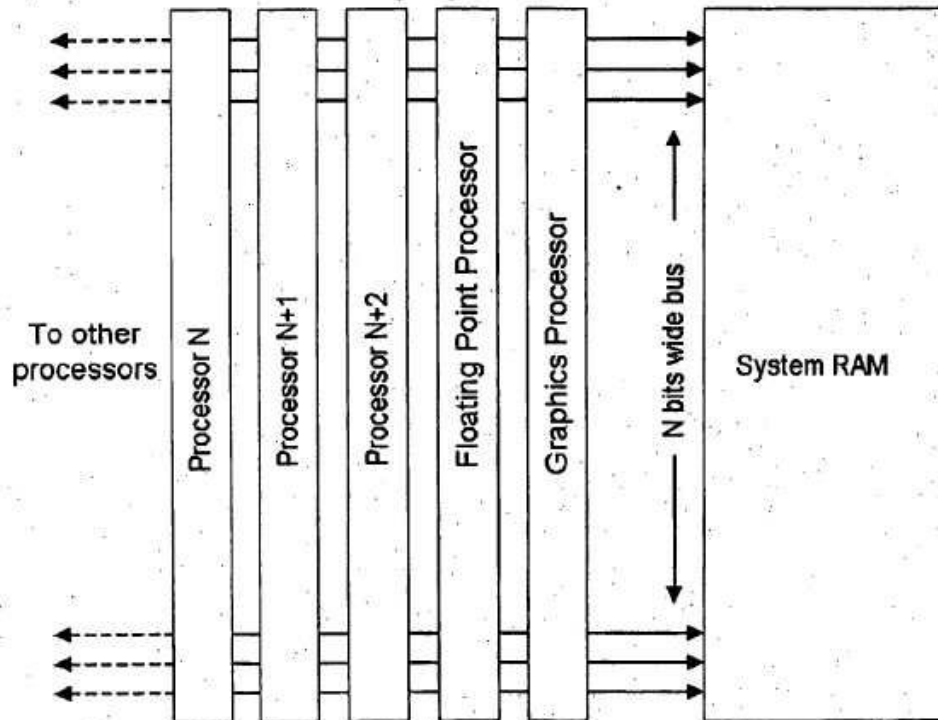
도면6

캐시 관리 상태

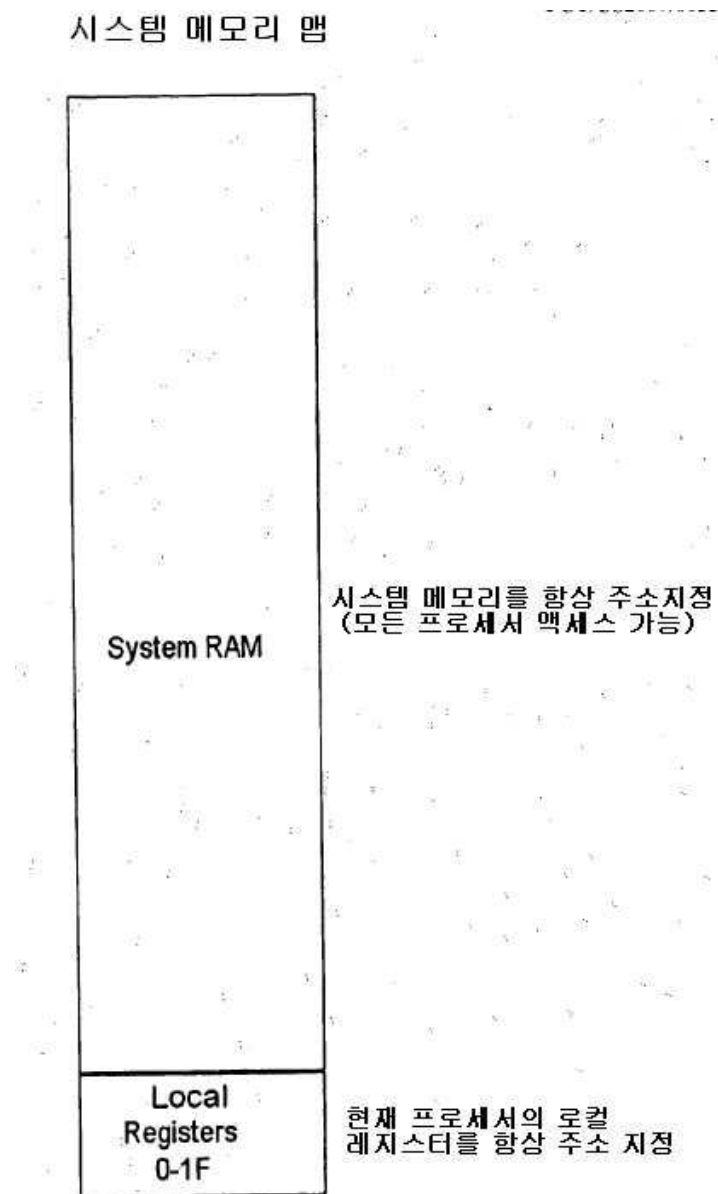


도면7

추가 기능



도면8



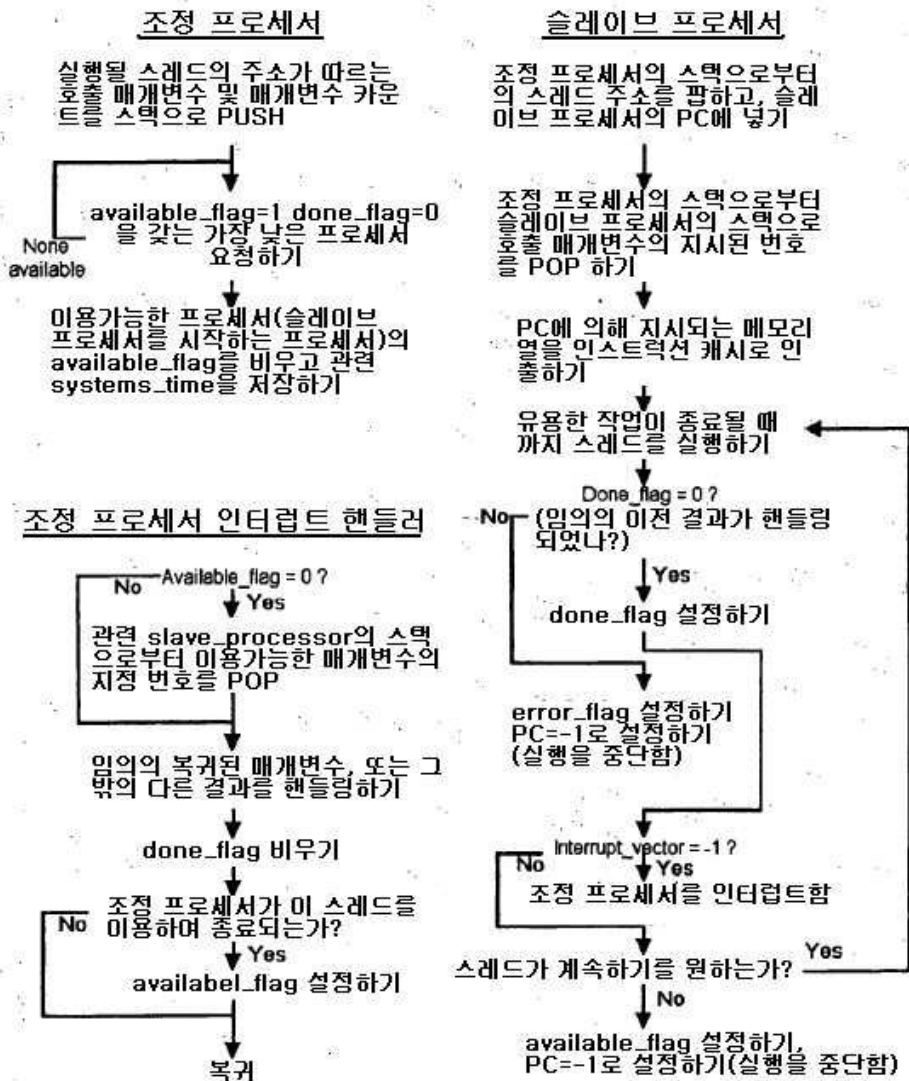
도면9

프로세서 이용가능성 표

Processor	available_flag	done_flag	Interrupt Vector	time_started	error_flag
3F					
3E					
3D					
3C					
3B					
3A					
30					
2F					
2E					
					
08					
07					
06					
05					
04					
03					
02					
01					

도면10

슬레이브 프로세서 할당 상태



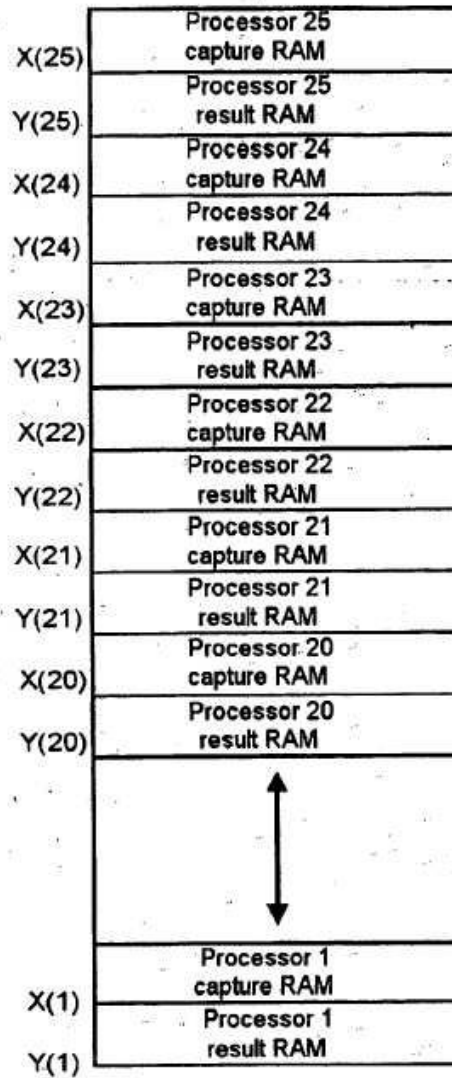
도면11

이미지 맵의 팩터링 컴퓨터 연산

processor 1	processor 2	processor 3	processor 4	processor 5	processor 6
processor 7	processor 8	processor 10	processor 11	processor 12	processor 13
processor 14	processor 15	processor 16	processor 17	processor 18	processor 19
processor 20	processor 21	processor 22	processor 23	processor 24	processor 25

도면12

팩터링 컴퓨터 연산 메모리 맵



도면13

64프로세서 시스템의 가능한 평면도

